

# Guía para defender el proyecto

## Inverso Multiplicativo Modular en x86-64 MASM

Documento de estudio para el equipo

### Índice

1. Idea general del proyecto
  2. Qué es el inverso multiplicativo modular
  3. Qué hace el programa completo
  4. Cómo se organiza un programa MASM
  5. Explicación de lo que está antes de `.code`
  6. Instrucciones básicas que aparecen en el código
  7. Registros importantes
  8. Cómo se pasan datos por la pila
  9. Explicación módulo por módulo
  10. Algoritmo extendido de Euclides explicado fácil
  11. Ejemplo completo con  $a = 3$  y  $p = 7$
  12. Preguntas probables de defensa
  13. Casos de prueba
  14. Resumen para estudiar rápido
  15. Fuentes consultadas
- 

### Objetivo de esta guía

Esta guía explica, de forma detallada pero sencilla, el código del proyecto de **Inverso Multiplicativo Modular** en ensamblador x86-64 usando MASM en Visual Studio.

La idea es que todos los integrantes puedan defender el proyecto aunque todavía no dominen todas las instrucciones de ensamblador. No se trata de memorizar línea por línea, sino de entender:

- qué hace el programa completo;
- qué hace cada módulo;
- cómo se pasan datos por la pila;
- por qué se usa el algoritmo extendido de Euclides;
- qué significan instrucciones como `mov`, `lea`, `push`, `call`, `cmp`, `idiv`, `QWORD PTR`, etc.;
- cómo responder preguntas del profesor durante la defensa.

**Frase corta para defensa:** El programa recibe dos enteros  $a$  y  $p$ , calcula si existe un número  $b$  tal que  $(a*b) \bmod p = 1$ , y muestra el inverso o un mensaje indicando que no existe.

## 1. Idea general del proyecto

El proyecto consiste en hacer un programa en ensamblador que calcule el **inverso multiplicativo modular** de un número entero  $a$  respecto a un módulo  $p$ .

El programa hace esto:

1. Pide al usuario el valor de  $a$ .
2. Pide al usuario el valor de  $p$ .
3. Calcula el máximo común divisor  $\text{mcd}(a, p)$ .
4. Si el  $\text{mcd}(a, p)$  es 1, entonces existe inverso.
5. Si existe inverso, calcula el valor y lo imprime.
6. Si no existe inverso, imprime un mensaje de error.

El código está dividido en procedimientos, llamados módulos en el proyecto. Esto ayuda a cumplir el requisito de dividir el programa y también hace que la defensa sea más ordenada.

| Parte del programa   | Responsabilidad                                                        |
|----------------------|------------------------------------------------------------------------|
| Entrada y salida     | Pedir datos al usuario y mostrar el resultado.                         |
| Control general      | Decidir qué módulo llamar y qué camino seguir.                         |
| Cálculo matemático   | Usar Euclides extendido para encontrar $\text{mcd}$ , $x$ y $y$ .      |
| Validación           | Revisar si vale la pena intentar calcular el inverso.                  |
| Ajuste del resultado | Convertir un resultado negativo a su equivalente positivo módulo $p$ . |

## 2. Qué es el inverso multiplicativo modular

El inverso multiplicativo modular de  $a$  módulo  $p$  es un número  $b$  que cumple:

$$(a * b) \bmod p = 1$$

Si encontramos ese número  $b$ , entonces  $b$  es el inverso de  $a$  módulo  $p$ .

## Ejemplo sencillo

Para:

$$\begin{aligned}a &= 3 \\ p &= 7\end{aligned}$$

El inverso es 5, porque:

$$\begin{aligned}3 * 5 &= 15 \\ 15 \bmod 7 &= 1\end{aligned}$$

Entonces:

$$3^{-1} \bmod 7 = 5$$

## Cuándo existe el inverso

El inverso existe solamente cuando  $a$  y  $p$  son coprimos, es decir, cuando:

$$\text{mcd}(a, p) = 1$$

Si el máximo común divisor es diferente de 1, el inverso no existe.

Ejemplo donde no existe:

$$\begin{aligned}a &= 4 \\ p &= 8 \\ \text{mcd}(4, 8) &= 4\end{aligned}$$

Como el mcd no es 1, no existe un número  $b$  que haga que:

$$(4 * b) \bmod 8 = 1$$

---

## 3. Qué hace el programa completo

El programa sigue este flujo:

```
main
├── leerDatos
├── inversoModular
│   ├── validarDatos
│   └── euclidesExtendido
├── ajustarPositivo
├── imprimirResultado
└── imprimirError
```

## Mapa de módulos

| Módulo            | Qué hace                                                     | Tipo              |
|-------------------|--------------------------------------------------------------|-------------------|
| main              | Coordina todo el programa. Llama a los demás procedimientos. | Control           |
| leerDatos         | Pide a y p usando printf y scanf.                            | Entrada           |
| imprimirResultado | Muestra el resultado cuando existe inverso.                  | Salida            |
| imprimirError     | Muestra el mensaje cuando no existe inverso.                 | Salida            |
| ajustarPositivo   | Convierte x a un valor positivo entre 0 y p-1.               | Cálculo           |
| validarDatos      | Revisa si p sirve y si a no es múltiplo de p.                | Validación        |
| euclidesExtendido | Calcula x, y y mcd.                                          | Cálculo           |
| inversoModular    | Une la validación con Euclides extendido.                    | Control / cálculo |

## 4. Cómo se organiza un programa MASM

El código está escrito en **MASM**, que es el ensamblador usado por Visual Studio para programas x64. Un programa MASM normalmente tiene una zona de datos y una zona de código.

| Elemento            | Significado                                                        |
|---------------------|--------------------------------------------------------------------|
| option casemap:none | Evita que MASM cambie mayúsculas/minúsculas de los nombres.        |
| includelib          | Indica qué bibliotecas se usarán al enlazar.                       |
| EXTERN              | Declara funciones que existen fuera del archivo .asm, como printf. |
| PUBLIC main         | Indica que main puede usarse como punto de entrada.                |
| .data               | Sección donde se guardan mensajes y formatos.                      |
| .code               | Sección donde empieza el código ejecutable.                        |
| PROC / ENDP         | Marca el inicio y el final de un procedimiento.                    |

| Elemento | Significado                              |
|----------|------------------------------------------|
| END      | Indica el final del archivo ensamblador. |

## 5. Explicación de lo que está antes de `.code`

Antes de `.code` aparece la configuración general del programa y los textos que se usarán para imprimir o leer desde consola.

```
option casemap:none  
option casemap:none
```

Esta línea le dice a MASM que respete las mayúsculas y minúsculas. Así, un nombre como `main` no se transforma automáticamente. Es una línea común en programas MASM.

### Bibliotecas usadas

```
includelib ucrt.lib  
includelib vcruntime.lib  
includelib legacy_stdio_definitions.lib  
includelib kernel32.lib
```

Estas líneas agregan bibliotecas externas al programa.

- `ucrt.lib`, `vcruntime.lib` y `legacy_stdio_definitions.lib` ayudan a usar funciones de C como `printf` y `scanf`.
- `kernel32.lib` permite usar funciones de Windows como `ExitProcess`.

En términos simples: estas bibliotecas permiten que nuestro ensamblador pueda usar funciones que no escribimos nosotros.

### Funciones externas

```
EXTERN printf:PROC  
EXTERN scanf:PROC  
EXTERN ExitProcess:PROC
```

`EXTERN` significa que esas funciones existen, pero no están escritas en nuestro archivo. El programa las buscará en las bibliotecas indicadas arriba.

- `printf` imprime texto en consola.
- `scanf` lee datos desde consola.
- `ExitProcess` termina el programa.

### Punto de entrada

```
PUBLIC main
```

Esto hace visible el procedimiento main. Visual Studio necesita saber desde dónde empieza el programa. En las propiedades del proyecto, el punto de entrada se configura como main.

## Sección .data

```
.data
msg_pedir_a    db "Ingrese el numero a: ", 0
msg_pedir_p    db "Ingrese el modulo p: ", 0
formato_num    db "%lld", 0

msg_resultado  db "El inverso de %lld modulo %lld es: %lld", 13, 10, 0
msg_error      db "El numero no tiene inverso multiplicativo modular",
13, 10, 0
```

La sección .data contiene datos fijos del programa.

### db

db significa **define byte**. Se usa para definir cadenas de texto byte por byte.

Ejemplo:

```
msg_pedir_a db "Ingrese el numero a: ", 0
```

Esto crea una cadena de texto llamada msg\_pedir\_a.

El 0 al final es importante porque las funciones de C como printf esperan cadenas terminadas en cero.

### %lld

```
formato_num db "%lld", 0
```

%lld es el formato que usa scanf para leer enteros largos de 64 bits.

### 13, 10

```
msg_resultado db "...", 13, 10, 0
```

Los valores 13 y 10 representan salto de línea en Windows.

- 13 = carriage return.
- 10 = line feed.

En la práctica, sirven para bajar a la siguiente línea después de imprimir.

---

## 6. Instrucciones básicas que aparecen en el código

Esta tabla explica las instrucciones principales que aparecen en el programa.

| Instrucción | Qué hace                                                  | Ejemplo del proyecto         |
|-------------|-----------------------------------------------------------|------------------------------|
| mov         | Copia un valor de un lugar a otro.                        | mov rax, QWORD PTR [rbp-8]   |
| lea         | Carga una dirección, no el valor guardado ahí.            | lea rax, [rbp-24]            |
| push        | Mete un valor en la pila.                                 | push QWORD PTR [rbp-8]       |
| call        | Llama a un procedimiento.                                 | call inversoModular          |
| add         | Suma o recupera espacio de la pila.                       | add rsp, 32                  |
| sub         | Resta o reserva espacio en la pila.                       | sub rsp, 80                  |
| cmp         | Compara dos valores.                                      | cmp rax, 1                   |
| je          | Salta si la comparación fue igual.                        | je calcular_inverso          |
| jne         | Salta si la comparación fue diferente.                    | jne main_no_existe           |
| jle         | Salta si es menor o igual.                                | jle validar_fin              |
| jge         | Salta si es mayor o igual.                                | jge ajustar_fin              |
| jmp         | Salto incondicional.                                      | jmp main_fin                 |
| xor         | Puede usarse para poner un registro en cero.              | xor ecx, ecx                 |
| cqo         | Extiende el signo de RAX hacia RDX: RAX antes de dividir. | cqo                          |
| idiv        | Divide con signo. Cociente en RAX, residuo en RDX.        | idiv QWORD PTR [rbp+24]      |
| imul        | Multiplica con signo.                                     | imul rax, QWORD PTR [rbp-32] |
| neg         | Cambia el signo de un número.                             | neg QWORD PTR [rbp-24]       |
| leave       | Deshace el stack frame.                                   | leave                        |
| ret         | Regresa al procedimiento que llamó.                       | ret                          |

## 7. Registros importantes

El código usa registros de 64 bits. Los más importantes son:

| Registro | Uso en el código                                                                                         |
|----------|----------------------------------------------------------------------------------------------------------|
| RAX      | Registro de trabajo. También recibe cociente en divisiones.                                              |
| RDX      | Segundo argumento para algunas llamadas y residuo después de <code>idiv</code> .                         |
| RCX      | Primer argumento para funciones externas como <code>printf</code> y <code>scanf</code> .                 |
| R8       | Tercer argumento para funciones externas.                                                                |
| R9       | Cuarto argumento para funciones externas.                                                                |
| RBP      | Base del stack frame. Permite acceder a parámetros y variables locales.                                  |
| RSP      | Puntero de la pila. Cambia cuando hacemos <code>push</code> , <code>sub</code> , <code>add</code> , etc. |

### Por qué `printf` y `scanf` usan `RCX`, `RDX`, `R8`, `R9`

En Windows x64, las funciones externas reciben sus primeros argumentos en registros. El orden es:

- 1.er argumento -> `RCX`
- 2.do argumento -> `RDX`
- 3.er argumento -> `R8`
- 4.to argumento -> `R9`

Por eso, aunque el proyecto pide que los módulos propios se comuniquen por pila, cuando se llama a `printf` o `scanf` sí se usan esos registros, porque así funciona la convención de llamadas de Windows x64.

---

## 8. Cómo se pasan datos por la pila

El proyecto pide que la comunicación entre los módulos propios sea por la pila. Por eso, antes de llamar un procedimiento, se usan instrucciones `push`.

Ejemplo de llamada a `inversoModular`:

```
push QWORD PTR [rbp-8]    ; a
push QWORD PTR [rbp-16]   ; p
```

```

lea rax, [rbp-24]
push rax                ; direccion de x

lea rax, [rbp-32]
push rax                ; direccion de mcd

call inversoModular
add rsp, 32

```

Se pasan 4 parámetros. Cada parámetro ocupa 8 bytes. Entonces, después del call, se limpia la pila con:

```
add rsp, 32
```

porque:

4 parametros \* 8 bytes = 32 bytes

## Stack frame

Cada procedimiento empieza normalmente así:

```

push rbp
mov rbp, rsp

```

Esto crea una base fija para acceder a parámetros y variables locales.

| Dirección                | Qué suele representar                |
|--------------------------|--------------------------------------|
| [rbp+16], [rbp+24], etc. | Parámetros recibidos por pila.       |
| [rbp-8], [rbp-16], etc.  | Variables locales del procedimiento. |
| [rbp]                    | RBP anterior guardado.               |
| [rbp+8]                  | Dirección de retorno.                |

## QWORD PTR explicado fácil

QWORD significa un dato de 8 bytes, o sea 64 bits. PTR le aclara a MASM que se está leyendo o escribiendo memoria de ese tamaño.

Ejemplo:

```
mov rax, QWORD PTR [rbp-8]
```

Significa:

Copiar a RAX el valor de 8 bytes guardado en [rbp-8].

Se usa mucho porque el programa trabaja en x64 y los enteros se manejan como valores de 64 bits.

---

## 9. Explicación módulo por módulo

### 9.1 main

main es el procedimiento principal. No hace toda la matemática directamente; coordina el programa y llama a los demás módulos.

#### Inicio de main

```
main PROC
    push rbp
    mov  rbp, rsp
    sub  rsp, 80
```

- main PROC inicia el procedimiento.
- push rbp guarda el valor anterior de RBP.
- mov rbp, rsp crea el stack frame.
- sub rsp, 80 reserva espacio para variables locales.

Variables locales de main:

| Variable | Significado              |
|----------|--------------------------|
| [rbp-8]  | a                        |
| [rbp-16] | p                        |
| [rbp-24] | x calculado por Euclides |
| [rbp-32] | mcd(a,p)                 |
| [rbp-40] | resultado final positivo |

#### Llamada a leerDatos

```
lea rax, [rbp-8]
push rax
```

```
lea rax, [rbp-16]
push rax
```

```
call leerDatos
add  rsp, 16
```

Aquí se pasan las direcciones donde se guardarán a y p.

Se usa lea porque se necesita la dirección de la variable, no su valor.

#### Llamada a inversoModular

```
push QWORD PTR [rbp-8]
push QWORD PTR [rbp-16]
```

```
lea rax, [rbp-24]
push rax
```

```
lea rax, [rbp-32]
push rax
```

```
call inversoModular
add rsp, 32
```

Aquí main manda:

- el valor de a;
- el valor de p;
- la dirección donde se guardará x;
- la dirección donde se guardará mcd.

#### Revisión del mcd

```
mov rax, QWORD PTR [rbp-32]
cmp rax, 1
jne main_no_existe
```

Se copia el mcd a RAX y se compara con 1.

- Si  $mcd = 1$ , sí existe inverso.
- Si  $mcd \neq 1$ , salta a main\_no\_existe.

#### Ajustar resultado positivo

```
push QWORD PTR [rbp-24]
push QWORD PTR [rbp-16]
```

```
lea rax, [rbp-40]
push rax
```

```
call ajustarPositivo
add rsp, 24
```

Euclides puede devolver un x negativo. Este módulo convierte ese valor a un número entre 0 y p-1.

#### Imprimir resultado

```
push QWORD PTR [rbp-8]
push QWORD PTR [rbp-16]
push QWORD PTR [rbp-40]
```

```
call imprimirResultado
add rsp, 24
```

Se mandan a, p y el resultado para imprimir.

#### Error y salida

```
main_no_existe:
    call imprimirError
```

```
main_fin:
    xor ecx, ecx
    call ExitProcess
```

Si no hay inverso, se imprime el error. Al final se llama a ExitProcess con código 0.

xor ecx, ecx pone ECX en cero.

---

## 9.2 leerDatos

Este módulo pide los dos números al usuario.

Recibe por pila:

| Dirección | Significado |
|-----------|-------------|
|-----------|-------------|

|          |                              |
|----------|------------------------------|
| [rbp+24] | Dirección donde se guarda a. |
|----------|------------------------------|

|          |                              |
|----------|------------------------------|
| [rbp+16] | Dirección donde se guarda p. |
|----------|------------------------------|

Inicio:

```
leerDatos PROC
    push rbp
    mov rbp, rsp
    and rsp, -16
```

and rsp, -16 alinea la pila a 16 bytes. Esto es importante para llamar funciones externas como printf y scanf.

### Imprimir mensaje para a

```
mov rcx, OFFSET msg_pedir_a
sub rsp, 32
call printf
add rsp, 32
```

- RCX recibe la dirección del mensaje.
- sub rsp, 32 reserva espacio requerido por la convención de llamadas.
- call printf imprime el mensaje.
- add rsp, 32 recupera el espacio.

### Leer a

```
mov rcx, OFFSET formato_num
mov rdx, QWORD PTR [rbp+24]
sub rsp, 32
call scanf
add rsp, 32
```

scanf necesita:

- formato en RCX;
- dirección donde guardar el número en RDX.

### Pedir y leer p

El mismo patrón se repite para p:

```
mov rcx, OFFSET msg_pedir_p
call printf
```

```
mov rcx, OFFSET formato_num
mov rdx, QWORD PTR [rbp+16]
call scanf
```

## 9.3 imprimirResultado

Este módulo imprime el resultado final.

Recibe por pila:

| Dirección | Significado |
|-----------|-------------|
| [rbp+32]  | a           |
| [rbp+24]  | p           |
| [rbp+16]  | resultado   |

Código importante:

```
mov rcx, OFFSET msg_resultado
mov rdx, QWORD PTR [rbp+32] ; a
mov r8, QWORD PTR [rbp+24] ; p
mov r9, QWORD PTR [rbp+16] ; resultado
```

```
sub rsp, 32
call printf
add rsp, 32
```

printf recibe:

```
RCX = formato
RDX = primer %lld
R8  = segundo %lld
R9  = tercer %lld
```

Por eso el mensaje:

El inverso de %lld modulo %lld es: %lld

usa tres valores.

---

## 9.4 imprimirError

Este módulo no recibe parámetros. Solo imprime el mensaje de error.

```
mov rcx, OFFSET msg_error
sub rsp, 32
call printf
add rsp, 32
```

Como solo imprime un mensaje, solo ocupa RCX.

---

## 9.5 ajustarPositivo

Este módulo toma el x que salió del algoritmo de Euclides y lo convierte a un resultado positivo.

Recibe:

| Dirección | Significado                          |
|-----------|--------------------------------------|
| [rbp+32]  | x                                    |
| [rbp+24]  | p                                    |
| [rbp+16]  | dirección donde guardar el resultado |

### División modular

```
mov rax, QWORD PTR [rbp+32]
cqo
idiv QWORD PTR [rbp+24]
```

```
mov rcx, rdx
```

- RAX recibe x.
- cqo prepara la división con signo.
- idiv p divide  $x / p$ .
- El residuo queda en RDX.
- El residuo es  $x \bmod p$ .

### Ajuste si es negativo

```
cmp rcx, 0
jge ajustar_fin
```

```
add rcx, QWORD PTR [rbp+24]
```

Si el residuo es negativo, se le suma p.

Ejemplo:

```
x = -2
p = 7
-2 mod 7 = 5
```

El resultado positivo es 5.

#### Guardar resultado

```
mov rax, QWORD PTR [rbp+16]
mov QWORD PTR [rax], rcx
```

La dirección donde se guarda el resultado está en [rbp+16]. Se copia esa dirección a RAX y luego se escribe el valor ahí.

---

## 9.6 validarDatos

Este módulo hace una validación inicial.

Recibe:

| Dirección | Significado                    |
|-----------|--------------------------------|
| [rbp+32]  | a                              |
| [rbp+24]  | p                              |
| [rbp+16]  | dirección donde guardar valido |

#### Inicializar valido = 0

```
mov rax, QWORD PTR [rbp+16]
mov QWORD PTR [rax], 0
```

Primero se asume que los datos no sirven.

#### Revisar que p > 1

```
mov rax, QWORD PTR [rbp+24]
cmp rax, 1
jle validar_fin
```

Si p <= 1, no se calcula inverso.

#### Revisar si a es múltiplo de p

```
mov rax, QWORD PTR [rbp+32]
cqo
idiv QWORD PTR [rbp+24]

cmp rdx, 0
je validar_fin
```

Aquí se calcula a mod p. Si el residuo es 0, entonces a es múltiplo de p, y no existe inverso.

```

Guardar valido = 1
mov rax, QWORD PTR [rbp+16]
mov QWORD PTR [rax], 1

```

Si pasó las pruebas, se guarda valido = 1.

---

## 9.7 euclidesExtendido

Este es el módulo matemático principal. Calcula valores x, y y mcd tales que:

$$a \cdot x + p \cdot y = \text{mcd}(a, p)$$

Si  $\text{mcd}(a, p) = 1$ , entonces x es el inverso modular de a módulo p.

### Parámetros que recibe

| Dirección | Significado                 |
|-----------|-----------------------------|
| [rbp+48]  | a                           |
| [rbp+40]  | p                           |
| [rbp+32]  | dirección donde guardar x   |
| [rbp+24]  | dirección donde guardar y   |
| [rbp+16]  | dirección donde guardar mcd |

### Variables locales

| Variable           | Significado             |
|--------------------|-------------------------|
| [rbp-8] = old_r    | Residuo anterior.       |
| [rbp-16] = r       | Residuo actual.         |
| [rbp-24] = old_s   | Coeficiente x anterior. |
| [rbp-32] = s       | Coeficiente x actual.   |
| [rbp-40] = old_t   | Coeficiente y anterior. |
| [rbp-48] = t       | Coeficiente y actual.   |
| [rbp-56] = q       | Cociente.               |
| [rbp-64] = residuo | Residuo temporal.       |

### Inicialización

```

old_r = a
r      = p
old_s = 1
s      = 0
old_t = 0
t      = 1

```

En código:

```
mov rax, QWORD PTR [rbp+48]
mov QWORD PTR [rbp-8], rax
```

```
mov rax, QWORD PTR [rbp+40]
mov QWORD PTR [rbp-16], rax
```

```
mov QWORD PTR [rbp-24], 1
mov QWORD PTR [rbp-32], 0
```

```
mov QWORD PTR [rbp-40], 0
mov QWORD PTR [rbp-48], 1
```

### Ciclo principal

ciclo\_euclides:

```
    cmp QWORD PTR [rbp-16], 0
    je  fin_euclides
```

El ciclo se repite mientras r no sea cero. Cuando r llega a cero, el mcd está en old\_r.

### División

```
mov rax, QWORD PTR [rbp-8]
cqo
idiv QWORD PTR [rbp-16]
```

```
mov QWORD PTR [rbp-56], rax
mov QWORD PTR [rbp-64], rdx
```

Se divide:

old\_r / r

Después:

- RAX tiene el cociente q.
- RDX tiene el residuo.

### Actualizar residuos

```
mov rax, QWORD PTR [rbp-16]
mov QWORD PTR [rbp-8], rax
```

```
mov rax, QWORD PTR [rbp-64]
mov QWORD PTR [rbp-16], rax
```

Esto equivale a:

old\_r = r  
r = residuo

### Actualizar coeficientes s

```
mov rax, QWORD PTR [rbp-56]
imul rax, QWORD PTR [rbp-32]
```

```
mov rcx, QWORD PTR [rbp-24]
sub rcx, rax
```

Esto calcula:

nuevo\_s = old\_s - q\*s

Luego se actualiza:

```
mov rax, QWORD PTR [rbp-32]
mov QWORD PTR [rbp-24], rax
```

```
mov QWORD PTR [rbp-32], rcx
```

Esto equivale a:

old\_s = s  
s = nuevo\_s

### Actualizar coeficientes t

Se hace igual que con s:

```
mov rax, QWORD PTR [rbp-56]
imul rax, QWORD PTR [rbp-48]
```

```
mov rcx, QWORD PTR [rbp-40]
sub rcx, rax
```

Esto calcula:

nuevo\_t = old\_t - q\*t

Después:

old\_t = t  
t = nuevo\_t

### Guardar resultados finales

Al final se guarda:

mcd = old\_r  
x = old\_s  
y = old\_t

En código:

```
mov rax, QWORD PTR [rbp+16]
mov rcx, QWORD PTR [rbp-8]
mov QWORD PTR [rax], rcx
```

```

mov rax, QWORD PTR [rbp+32]
mov rcx, QWORD PTR [rbp-24]
mov QWORD PTR [rax], rcx

mov rax, QWORD PTR [rbp+24]
mov rcx, QWORD PTR [rbp-40]
mov QWORD PTR [rax], rcx

```

---

## 9.8 inversoModular

Este módulo conecta todo el cálculo.

Recibe:

| Dirección | Significado      |
|-----------|------------------|
| [rbp+40]  | a                |
| [rbp+32]  | p                |
| [rbp+24]  | dirección de x   |
| [rbp+16]  | dirección de mcd |

Variables locales:

| Variable | Significado |
|----------|-------------|
| [rbp-8]  | valido      |
| [rbp-16] | y           |

### Llamar a validarDatos

```

push QWORD PTR [rbp+40]
push QWORD PTR [rbp+32]

```

```

lea rax, [rbp-8]
push rax

```

```

call validarDatos
add rsp, 24

```

Se pasan a, p y la dirección de valido.

### Revisar si se puede calcular

```

cmp QWORD PTR [rbp-8], 1
je calcular_inverso

```

Si valido = 1, se calcula el inverso. Si no, se deja  $x = 0$  y  $mcd = 0$ .

### Llamar a Euclides

```

push QWORD PTR [rbp+40]
push QWORD PTR [rbp+32]

```

```
mov rax, QWORD PTR [rbp+24]
push rax
```

```
lea rax, [rbp-16]
push rax
```

```
mov rax, QWORD PTR [rbp+16]
push rax
```

```
call euclidesExtendido
add rsp, 40
```

Se pasan:

- a;
  - p;
  - dirección de x;
  - dirección de y;
  - dirección de mcd.
- 

## 10. Algoritmo extendido de Euclides explicado fácil

El algoritmo de Euclides normal encuentra el máximo común divisor usando divisiones repetidas.

El algoritmo extendido hace lo mismo, pero además encuentra dos números  $x$  y  $y$  que cumplen:

$$a \cdot x + p \cdot y = \text{mcd}(a, p)$$

Si el mcd es 1:

$$a \cdot x + p \cdot y = 1$$

Si vemos esa igualdad módulo  $p$ , el término  $p \cdot y$  desaparece porque es múltiplo de  $p$ .

Entonces queda:

$$a \cdot x \bmod p = 1$$

Por eso  $x$  es el inverso modular.

**Forma fácil de decirlo:** Euclides extendido encuentra un  $x$  que multiplica a  $a$ . Si el mcd es 1, ese  $x$  cumple la condición del inverso modular. Luego el programa lo ajusta para que sea positivo.

---

## 11. Ejemplo completo con $a = 3$ y $p = 7$

El usuario ingresa:

$a = 3$

$p = 7$

El flujo es:

1. `main` llama a `leerDatos`.
2. `leerDatos` guarda  $a = 3$  y  $p = 7$ .
3. `main` llama a `inversoModular`.
4. `inversoModular` llama a `validarDatos`.
5. `validarDatos` confirma que  $p > 1$  y que  $3 \bmod 7$  no es 0.
6. `inversoModular` llama a `euclidesExtendido`.
7. `Euclides` calcula que  $\text{mcd}(3, 7) = 1$ .
8. `Euclides` puede encontrar  $x = -2$ , porque:  
 $3*(-2) + 7*(1) = 1$

Ese  $x = -2$  sirve, pero no está en el rango positivo. Entonces:

$$-2 \bmod 7 = 5$$

Por eso el programa imprime:

El inverso de 3 modulo 7 es: 5

---

## 12. Preguntas probables de defensa

### ¿Por qué usan el algoritmo extendido de Euclides?

Porque permite calcular el `mcd` y además encontrar  $x$  y  $y$  tales que:

$$a*x + p*y = \text{mcd}(a, p)$$

Si el `mcd` es 1, entonces  $x$  es el inverso modular.

### ¿Por qué si $\text{mcd}(a, p)$ no es 1 no existe inverso?

Porque para que exista un número  $b$  con:

$$(a*b) \bmod p = 1$$

$a$  y  $p$  deben ser coprimos. Eso significa que el `mcd` debe ser 1.

### ¿Qué hace `main`?

Coordina el programa: lee los datos, llama al cálculo, revisa el mcd y decide si imprime resultado o error.

### ¿Qué hace `inversoModular`?

Primero valida los datos. Si son válidos, llama a `euclidesExtendido` para obtener  $x$  y  $mcd$ .

### ¿Qué hace `ajustarPositivo`?

Convierte el  $x$  que da Euclides a un número entre  $0$  y  $p-1$ , porque  $x$  puede salir negativo.

### ¿Por qué usan `QWORD PTR`?

Porque los valores son de 64 bits. `QWORD` indica 8 bytes y `PTR` aclara que se está accediendo a memoria.

### ¿Por qué usan `RCX, RDX, R8 y R9` en `printf`?

Porque las funciones externas en Windows x64 reciben sus primeros argumentos en esos registros.

### ¿Por qué usan `push` para llamar sus módulos?

Porque el proyecto pide que la comunicación entre módulos propios sea por pila.

### ¿Por qué aparece `sub rsp, 32` antes de `printf` o `scanf`?

Porque en Windows x64 se reserva espacio en pila para llamadas a funciones externas.

### ¿Por qué se usa `and rsp, -16`?

Para alinear la pila a 16 bytes antes de llamar funciones externas como `printf` o `scanf`.

---

## 13. Casos de prueba

| a  | p  | Resultado esperado |
|----|----|--------------------|
| 3  | 7  | 5                  |
| 4  | 8  | No tiene inverso   |
| 10 | 17 | 12                 |
| 2  | 5  | 3                  |

| a | p  | Resultado esperado |
|---|----|--------------------|
| 5 | 11 | 9                  |
| 7 | 13 | 2                  |
| 6 | 9  | No tiene inverso   |
| 1 | 7  | 1                  |
| 7 | 7  | No tiene inverso   |

Para defenderlo, lo más importante es probar al menos un caso donde sí existe inverso y otro donde no existe.

---

## 14. Resumen para estudiar rápido

- El programa calcula el inverso modular de  $a$  módulo  $p$ .
  - El inverso existe si  $\text{mcd}(a, p) = 1$ .
  - Euclides extendido calcula  $x$ ,  $y$  y  $\text{mcd}$ .
  - Si  $\text{mcd} = 1$ ,  $x$  es el inverso modular, aunque puede salir negativo.
  - `ajustarPositivo` convierte  $x$  al rango  $0 \dots p-1$ .
  - Los módulos del proyecto se comunican por pila usando `push` y `call`.
  - `printf` y `scanf` usan registros porque son funciones externas de Windows x64.
  - `QWORD PTR` se usa para leer o escribir datos de 64 bits en memoria.
  - `RBP` sirve como base fija para acceder a parámetros y variables locales.
  - `RSP` cambia cuando se mete o se saca información de la pila.
- 

## 15. Fuentes consultadas

Estas fuentes ayudan a justificar detalles de MASM y de la convención de llamadas de Windows x64:

- Microsoft Docs - MASM for x64 (`ml64.exe`): <https://learn.microsoft.com/en-us/cpp/asm/assembler/masm/masm-for-x64-ml64-exe>
- Microsoft Docs - x64 calling convention: <https://learn.microsoft.com/en-us/cpp/build/x64-calling-convention>
- Microsoft Docs - MASM operator `PTR`: <https://learn.microsoft.com/en-us/cpp/asm/assembler/masm/operator-ptr>